

16. Controller Area Network - CAN

The controller area network (CAN) protocol is a real-time, serial, broadcast protocol with a very high level of security. The ATmega16/32/64/M1/C1 CAN controller is fully compatible with the CAN Specification 2.0 Part A and Part B. It delivers the features required to implement the kernel of the CAN bus protocol according to the ISO/OSI reference model:

- The data link layer
 - the logical link control (LLC) sublayer
 - the medium access control (MAC) sublayer
- The physical layer
 - the physical signalling (PLS) sublayer
 - not supported - the physical medium attach (PMA)
 - not supported - the medium dependent interface (MDI)

The CAN controller is able to handle all types of frames (data, remote, error and overload) and achieves a bitrate of 1Mbit/s.

16.1 Features

- Full can controller
- Fully compliant with CAN standard rev 2.0 A and rev 2.0 B
- 6 MOB (message object) with their own:
 - 11 bits of identifier tag (rev 2.0 A), 29 bits of identifier tag (rev 2.0 B)
 - 11 bits of identifier mask (rev 2.0 A), 29 bits of identifier mask (rev 2.0 B)
 - 8 bytes data buffer (static allocation)
 - Tx, Rx, frame buffer or automatic reply configuration
 - Time stamping
- 1Mbit/s maximum transfer rate at 8MHz
- TTC timer
- Listening mode (for spying or autobaud)

16.2 CAN Protocol

The CAN protocol is an international standard defined in the ISO 11898 for high speed and ISO 11519-2 for low speed.

16.2.1 Principles

CAN is based on a broadcast communication mechanism. This broadcast communication is achieved by using a message oriented transmission protocol. These messages are identified by using a message identifier. Such a message identifier has to be unique within the whole network and it defines not only the content but also the priority of the message.

The priority at which a message is transmitted compared to another less urgent message is specified by the identifier of each message. The priorities are laid down during system design in the form of corresponding binary values and cannot be changed dynamically. The identifier with the lowest binary number has the highest priority.

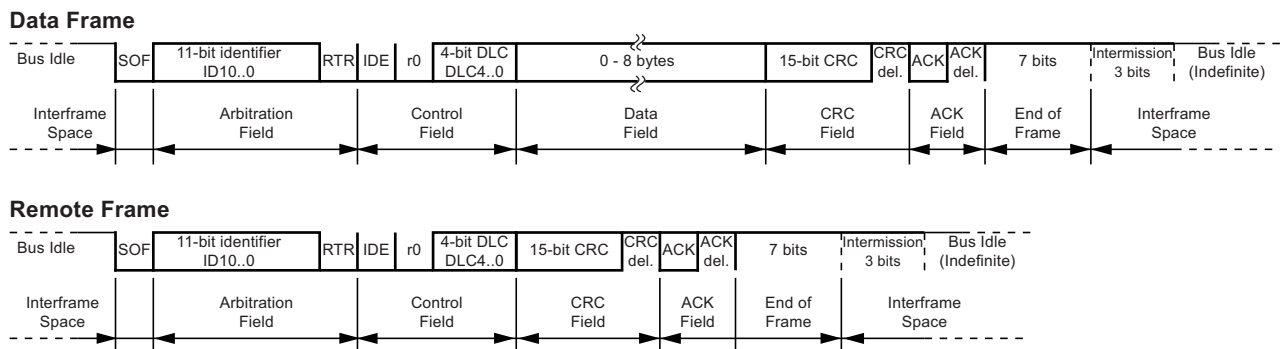
Bus access conflicts are resolved by bit-wise arbitration on the identifiers involved by each node observing the bus level bit for bit. This happens in accordance with the “wired and” mechanism, by which the dominant state overwrites the recessive state. The competition for bus allocation is lost by all nodes with recessive transmission and dominant observation. All the “losers” automatically become receivers of the message with the highest priority and do not re-attempt transmission until the bus is available again.

16.2.2 Message Formats

The CAN protocol supports two message frame formats, the only essential difference being in the length of the identifier. The CAN standard frame, also known as CAN 2.0 A, supports a length of 11 bits for the identifier, and the CAN extended frame, also known as CAN 2.0 B, supports a length of 29 bits for the identifier.

16.2.2.1 Can Standard Frame

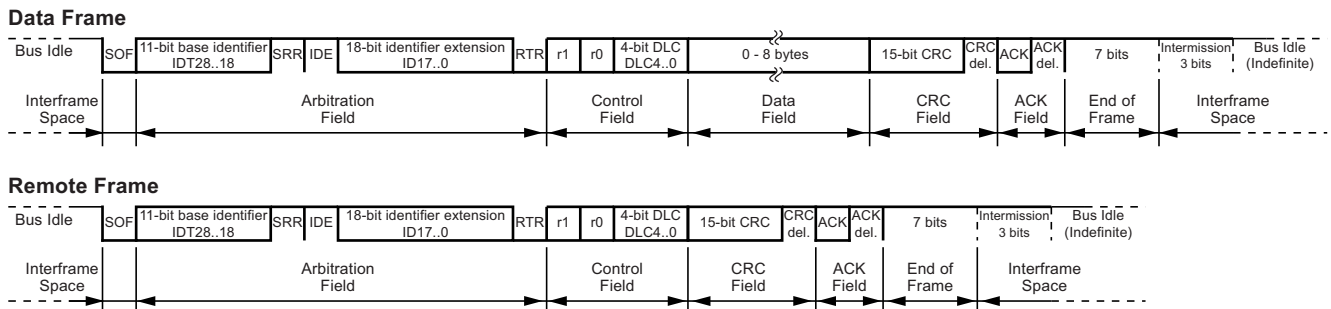
Figure 16-1. CAN Standard Frames



A message in the CAN standard frame format begins with the “Start Of Frame (SOF)”, this is followed by the “Arbitration field” which consist of the identifier and the “Remote Transmission Request (RTR)” bit used to distinguish between the data frame and the data request frame called remote frame. The following “Control field” contains the “IDentifier Extension (IDE)” bit and the “Data Length Code (DLC)” used to indicate the number of following data bytes in the “Data field”. In a remote frame, the DLC contains the number of requested data bytes. The “Data field” that follows can hold up to 8 data bytes. The frame integrity is guaranteed by the following “Cyclic Redundant Check (CRC)” sum. The “ACKnowledge (ACK) field” comprises the ACK slot and the ACK delimiter. The bit in the ACK slot is sent as a recessive bit and is overwritten as a dominant bit by the receivers which have at this time received the data correctly. Correct messages are acknowledged by the receivers regardless of the result of the acceptance test. The end of the message is indicated by “End Of Frame (EOF)”. The “Intermission Frame Space (IFS)” is the minimum number of bits separating consecutive messages. If there is no following bus access by any node, the bus remains idle.

16.2.2.2 CAN Extended Frame

Figure 16-2. CAN Extended Frames



A message in the CAN extended frame format is likely the same as a message in CAN standard frame format. The difference is the length of the identifier used. The identifier is made up of the existing 11-bit identifier (base identifier) and an 18-bit extension (identifier extension). The distinction between CAN standard frame format and CAN extended frame format is made by using the IDE bit which is transmitted as dominant in case of a frame in CAN standard frame format, and transmitted as recessive in the other case.

16.2.2.3 Format Co-existence

As the two formats have to co-exist on one bus, it is laid down which message has higher priority on the bus in the case of bus access collision with different formats and the same identifier / base identifier: The message in CAN standard frame format always has priority over the message in extended format.

There are three different types of CAN modules available:

- 2.0A - considers 29 bit ID as an error
- 2.0B passive - ignores 29 bit ID messages
- 2.0B active - handles both 11 and 29 bit ID messages

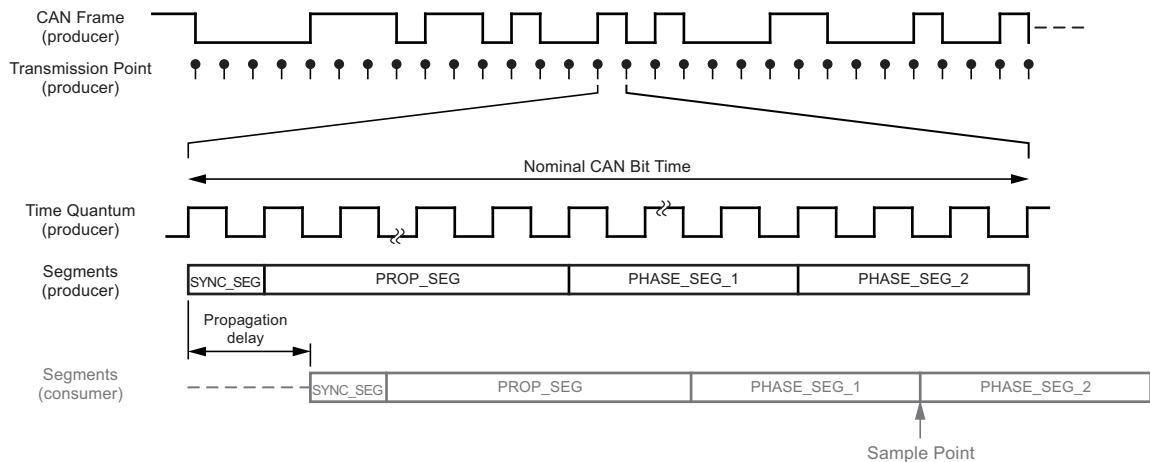
16.2.3 CAN Bit Timing

To ensure correct sampling up to the last bit, a CAN node needs to re-synchronize throughout the entire frame. This is done at the beginning of each message with the falling edge SOF and on each recessive to dominant edge.

16.2.3.1 Bit Construction

One CAN bit time is specified as four non-overlapping time segments. Each segment is constructed from an integer multiple of the time quantum. The time quantum or TQ is the smallest discrete timing resolution used by a CAN node.

Figure 16-3. CAN Bit Construction



16.2.3.2 Synchronization Segment

The first segment is used to synchronize the various bus nodes.

On transmission, at the start of this segment, the current bit level is output. If there is a bit state change between the previous bit and the current bit, then the bus state change is expected to occur within this segment by the receiving nodes.

16.2.3.3 Propagation Time Segment

This segment is used to compensate for signal delays across the network.

This is necessary to compensate for signal propagation delays on the bus line and through the transceivers of the bus nodes.

16.2.3.4 Phase Segment 1

Phase Segment 1 is used to compensate for edge phase errors.

This segment may be lengthened during re-synchronization.

16.2.3.5 Sample Point

The sample point is the point of time at which the bus level is read and interpreted as the value of the respective bit. Its location is at the end of phase segment 1 (between the two phase segments).

16.2.3.6 Phase Segment 2

This segment is also used to compensate for edge phase errors.

This segment may be shortened during re-synchronization, but the length has to be at least as long as the information processing time (IPT) and may not be more than the length of phase segment 1.

16.2.3.7 Information Processing Time

It is the time required for the logic to determine the bit level of a sampled bit.

The IPT begins at the sample point, is measured in TQ and is fixed at 2TQ for the Atmel CAN. Since phase segment 2 also begins at the sample point and is the last segment in the bit time, PS2 minimum shall not be less than the IPT.

16.2.3.8 Bit Lengthening

As a result of resynchronization, phase segment 1 may be lengthened or phase segment 2 may be shortened to compensate for oscillator tolerances. If, for example, the transmitter oscillator is slower than the receiver oscillator, the next falling edge used for resynchronization may be delayed. So phase segment 1 is lengthened in order to adjust the sample point and the end of the bit time.

16.2.3.9 Bit Shortening

If, on the other hand, the transmitter oscillator is faster than the receiver one, the next falling edge used for resynchronization may be too early. So phase segment 2 in bit N is shortened in order to adjust the sample point for bit N+1 and the end of the bit time

16.2.3.10 Synchronization Jump Width

The limit to the amount of lengthening or shortening of the phase segments is set by the Resynchronization jump width. This segment may not be longer than phase segment 2.

16.2.3.11 Programming the Sample Point

Programming of the sample point allows “tuning” of the characteristics to suit the bus.

Early sampling allows more time quanta in the phase segment 2 so the synchronization jump width can be programmed to its maximum. This maximum capacity to shorten or lengthen the bit time decreases the sensitivity to node oscillator tolerances, so that lower cost oscillators such as ceramic resonators may be used.

Late sampling allows more time quanta in the propagation time segment which allows a poorer bus topology and maximum bus length.

16.2.3.12 Synchronization

Hard synchronization occurs on the recessive-to-dominant transition of the start bit. The bit time is restarted from that edge.

Re-synchronization occurs when a recessive-to-dominant edge doesn't occur within the synchronization segment in a message.

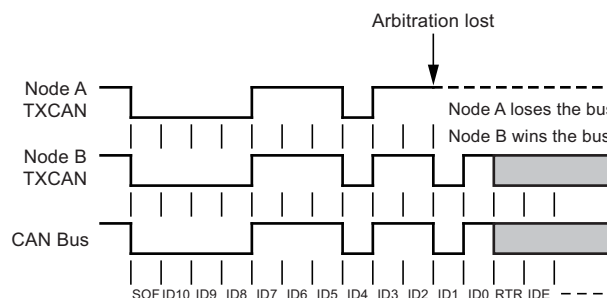
16.2.4 Arbitration

The CAN protocol handles bus accesses according to the concept called “Carrier Sense Multiple Access with Arbitration on Message Priority”.

during transmission, arbitration on the CAN bus can be lost to a competing device with a higher priority CAN Identifier. This arbitration concept avoids collisions of messages whose transmission was started by more than one node simultaneously and makes sure the most important message is sent first without time loss.

The bus access conflict is resolved during the arbitration field mostly over the identifier value. If a data frame and a remote frame with the same identifier are initiated at the same time, the data frame prevails over the remote frame (c.f. RTR bit).

Figure 16-4. Bus Arbitration



16.2.5 Errors

The CAN protocol signals any errors immediately as they occur. Three error detection mechanisms are implemented at the message level and two at the bit level:

16.2.5.1 Error at Message Level

- **Cyclic redundancy check (CRC)**
The CRC safeguards the information in the frame by adding redundant check bits at the transmission end. At the receiver these bits are re-computed and tested against the received bits. If they do not agree there has been a CRC error.
- **Frame check**
This mechanism verifies the structure of the transmitted frame by checking the bit fields against the fixed format and the frame size. Errors detected by frame checks are designated “format errors”.
- **ACK errors**
As already mentioned frames received are acknowledged by all receivers through positive acknowledgement. If no acknowledgement is received by the transmitter of the message an ACK error is indicated.

16.2.5.2 Error at Bit Level

- **Monitoring**
The ability of the transmitter to detect errors is based on the monitoring of bus signals. Each node which transmits also observes the bus level and thus detects differences between the bit sent and the bit received. This permits reliable detection of global errors and errors local to the transmitter.
- **Bit stuffing**
The coding of the individual bits is tested at bit level. The bit representation used by CAN is “Non Return to Zero (NRZ)” coding, which guarantees maximum efficiency in bit coding. The synchronization edges are generated by means of bit stuffing.

16.2.5.3 Error Signalling

If one or more errors are discovered by at least one node using the above mechanisms, the current transmission is aborted by sending an “error flag”. This prevents other nodes accepting the message and thus ensures the consistency of data throughout the network. After transmission of an erroneous message that has been aborted, the sender automatically re-attempts transmission.

16.3 CAN Controller

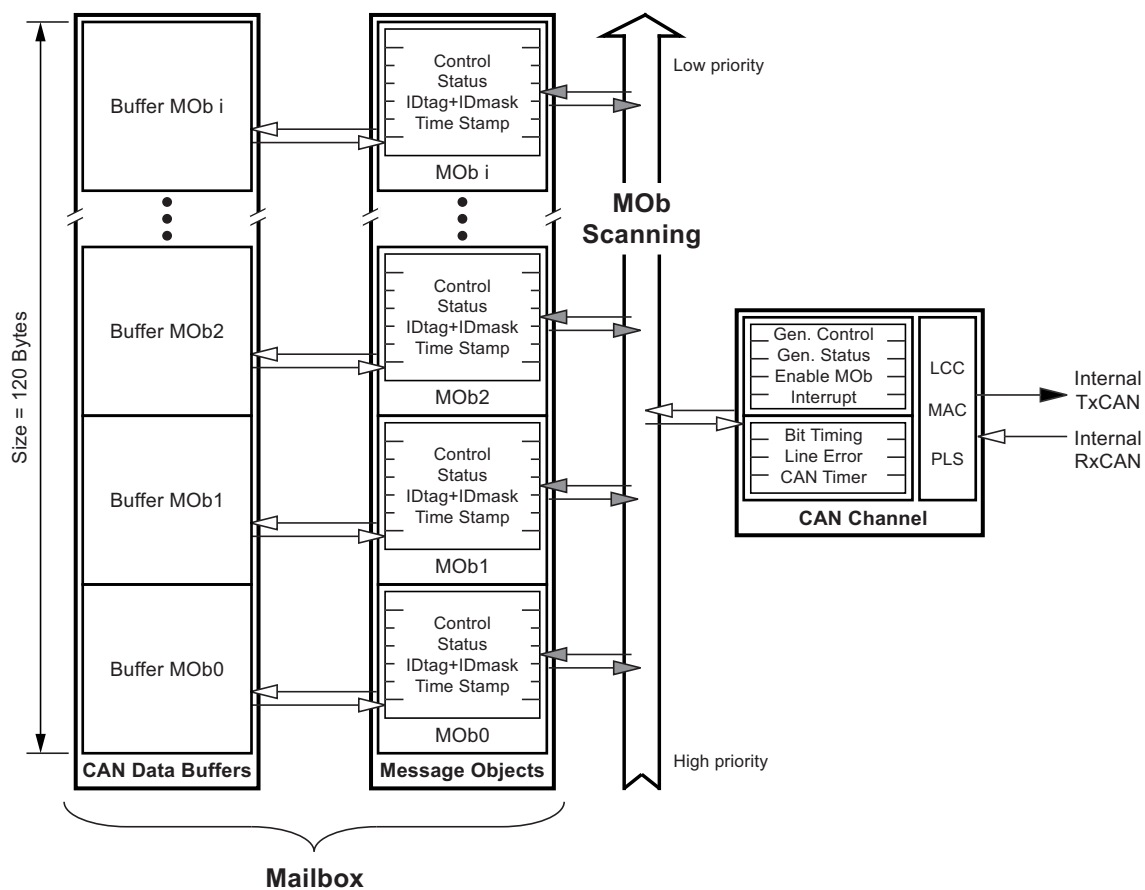
The CAN controller implemented into ATmega16/32/64/M1/C1 offers V2.0B active.

This full-CAN controller provides the whole hardware for convenient acceptance filtering and message management. For each message to be transmitted or received this module contains one so called message object in which all information regarding the message (e.g. identifier, data bytes etc.) are stored.

During the initialization of the peripheral, the application defines which messages are to be sent and which are to be received. Only if the CAN controller receives a message whose identifier matches with one of the identifiers of the programmed (receive) message objects the message is stored and the application is informed by interrupt. Another advantage is that incoming remote frames can be answered automatically by the full-CAN controller with the corresponding data frame. In this way, the CPU load is strongly reduced compared to a basic-CAN solution.

Using full-CAN controller, high baudrates and high bus loads with many messages can be handled.

Figure 16-5. CAN Controller Structure



16.4 CAN Channel

16.4.1 Configuration

The CAN channel can be in:

- Enabled mode

In this mode:

- the CAN channel (internal TxCAN and RxCAN) is enabled,
- the input clock is enabled.

- Standby mode

In standby mode:

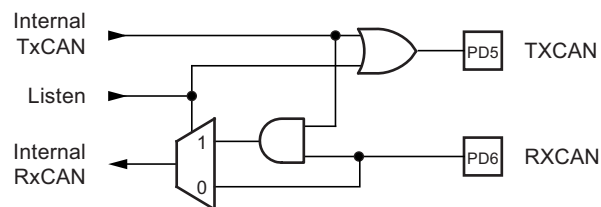
- the transmitter constantly provides a recessive level (on internal TxCAN) and the receiver is disabled,
- input clock is enabled,
- the registers and pages remain accessible.

- Listening mode

This mode is transparent for the CAN channel:

- enables a hardware loop back, internal TxCAN on internal RxCAN
- provides a recessive level on TXCAN output pin
- does not disable RXCAN input pin
- freezes TEC and REC error counters

Figure 16-6. Listening Mode



16.4.2 Bit Timing

FSM's (finite state machine) of the CAN channel need to be synchronous to the time quantum. So, the input clock for bit timing is the clock used into CAN channel FSM's.

Field and segment abbreviations:

- BRP: Baud rate prescaler.
- TQ: Time quantum (output of baud rate prescaler).
- SYNS: Synchronization segment is 1 TQ long.
- PRS: Propagation time segment is programmable to be 1, 2, ..., 8 TQ long.
- PHS1: Phase segment 1 is programmable to be 1, 2, ..., 8 TQ long.
- PHS2: Phase segment 2 is programmable to be $\leq$ PHS1 and $\geq$ INFORMATION PROCESSING TIME.
- INFORMATION PROCESSING TIME is 2 TQ.
- SJW: (Re) Synchronization jump width is programmable between 1 and min(4, PHS1).

The total number of TQ in a bit time has to be programmed at least from 8 to 25.

Figure 16-7. Sample and Transmission Point

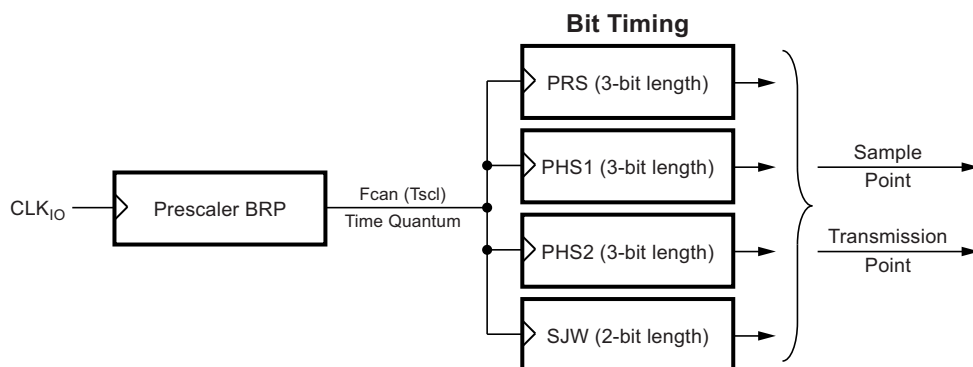
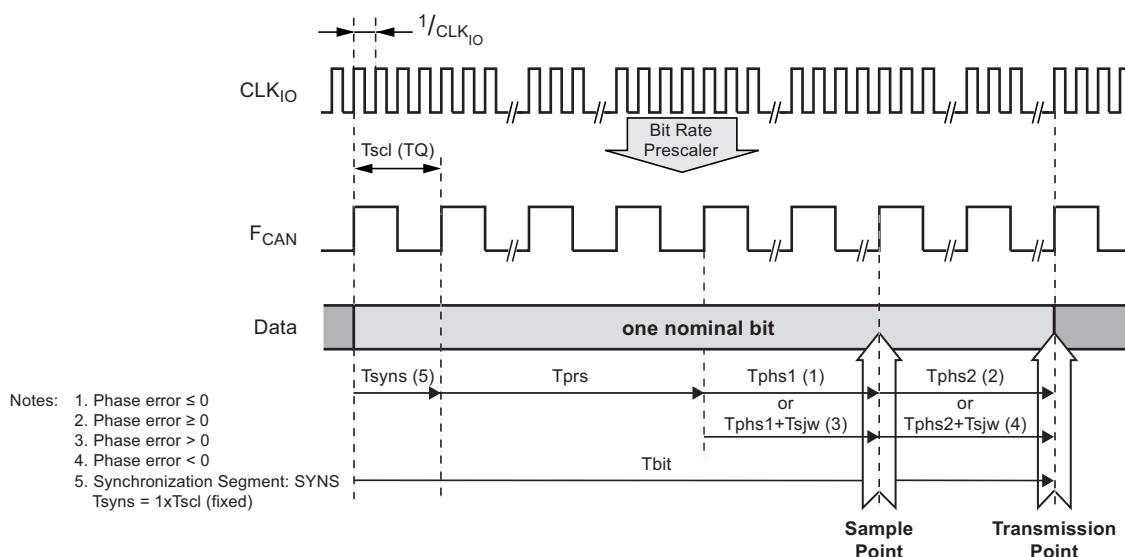


Figure 16-8. General Structure of a Bit Period



16.4.3 Baud Rate

With no baud rate prescaler (BRP[5..0]=0) the sampling point comes one time quantum too early. This leads to a fail according the ISO16845 Test plan. It is necessary to lengthen the phase segment 1 by one time quantum and to shorten the phase segment 2 by one time quantum to compensate.

The baud rate selection is made by T_{bit} calculation:

$$T_{bit}^{(1)} = T_{syns} + T_{prs} + T_{phs1} + T_{phs2}$$

1. $T_{syns} = 1 \times T_{scL} = (BRP[5..0] + 1) / clk_{IO} (= 1TQ)$
2. $T_{prs} = (1 \text{ to } 8) \times T_{scL} = (PRS[2..0] + 1) \times T_{scL}$
3. $T_{phs1} = (1 \text{ to } 8) \times T_{scL} = (PHS1[2..0] + 1) \times T_{scL}$
4. $T_{phs2} = (1 \text{ to } 8) \times T_{scL} = (PHS2[2..0]^{(2)} + 1) \times T_{scL}$
5. $T_{sjw} = (1 \text{ to } 4) \times T_{scL} = (SJW[1..0] + 1) \times T_{scL}$

- Notes:
1. The total number of T_{scL} (Time Quanta) in a bit time must be from 8 to 25.
 2. $PHS2[2..0]$ 2 is programmable to be $\leq PHS1[2..0]$ and ≥ 1 .

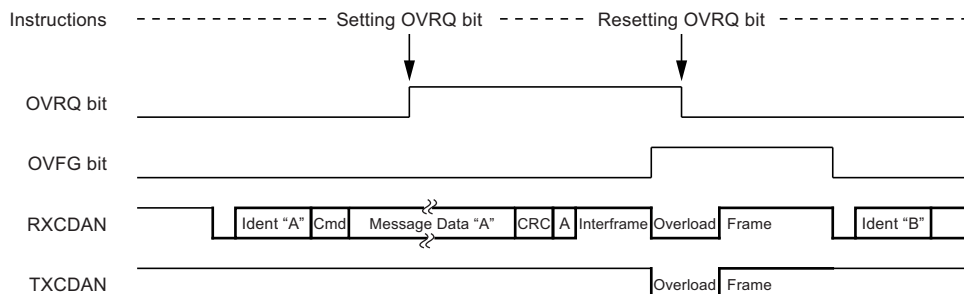
16.4.4 Fault Confinement

(c.f. [Section 16.7 "Error Management" on page 153](#)).

16.4.5 Overload Frame

An overload frame is sent by setting an overload request (OVRQ). After the next reception, the CAN channel sends an overload frame in accordance with the CAN specification. A status or flag is set (OVRF) as long as the overload frame is sent.

Figure 16-9. Overload Frame



16.5 Message Objects

The MOB is a CAN frame descriptor. It contains all information to handle a CAN frame. This means that a MOB has been outlined to allow to describe a CAN message like an object. The set of MOBs is the front end part of the “mailbox” where the messages to send and/or to receive are pre-defined as well as possible to decrease the work load of the software.

The MOBs are independent but priority is given to the lower one in case of multi matching. The operating modes are:

- Disabled mode
- Transmit mode
- Receive mode
- Automatic reply
- Frame buffer receive mode

16.5.1 Number of MOBs

This device has 6 MOBs, they are numbered from 0 up to 5 (i=5).

16.5.2 Operating Modes

There is **no** default mode after RESET.

Every MOB has its own fields to control the operating mode. Before enabling the CAN peripheral, each MOB must be configured (ex: disabled mode - CONMOB=00).

Table 16-1. MOB Configuration

MOB Configuration		Reply Valid	RTR Tag	Operating Mode
0	0	x	x	Disabled
0	1	x	0	Tx Data Frame
		x	1	Tx Remote Frame
1	0	x	0	Rx Data Frame
		0	1	Rx Remote Frame
		1		Rx Remote Frame then, Tx Data Frame (reply)
1	1	x	x	Frame Buffer Receive Mode

16.5.2.1 Disabled

In this mode, the MOB is “free”.

16.5.2.2 Tx Data and Remote Frame

1. Several fields must be initialized before sending:
 - Identifier tag (IDT)
 - Identifier extension (IDE)
 - Remote transmission request (RTRTAG)
 - Data length code (DLC)
 - Reserved bit(s) tag (RBnTAG)
 - Data bytes of message (MSG)
2. The MOB is ready to send a data or a remote frame when the MOB configuration is set (CONMOB).
3. Then, the CAN channel scans all the MOBs in Tx configuration, finds the MOB having the highest priority and tries to send it.
4. When the transmission is completed the TXOK flag is set (interrupt).
5. All the parameters and data are available in the MOB until a new initialization.

16.5.2.3 Rx Data and Remote Frame

1. Several fields must be initialized before receiving:
 - Identifier tag (IDT)
 - Identifier mask (IDMSK)
 - Identifier extension (IDE)
 - Identifier extension mask (IDEMSK)
 - Remote transmission request (RTRTAG)
 - Remote transmission request mask (RTRMSK)
 - Data length code (DLC)
 - Reserved bit(s) tag (RBnTAG)
2. The MOB is ready to receive a data or a remote frame when the MOB configuration is set (CONMOB).
3. When a frame identifier is received on CAN network, the CAN channel scans all the MOBs in receive mode, tries to find the MOB having the highest priority which is matching.
4. On a hit, the IDT, the IDE and the DLC of the matched MOB are updated from the incoming (frame) values.
5. Once the reception is completed, the data bytes of the received message are stored (not for remote frame) in the data buffer of the matched MOB and the RXOK flag is set (interrupt).
6. All the parameters and data are available in the MOB until a new initialization.

16.5.2.4 Automatic Reply

A reply (data frame) to a remote frame can be automatically sent after reception of the expected remote frame.

1. Several fields must be initialized before receiving the remote frame:
 - Reply valid (RPLV) in a identical flow to the one described in [Section 16.5.2.3 “Rx Data and Remote Frame” on page 150](#).
2. When a remote frame matches, automatically the RTRTAG and the reply valid bit (RPLV) are reset. No flag (or interrupt) is set at this time. Since the CAN data buffer has not been used by the incoming remote frame, the MOB is then ready to be in transmit mode without any more setting. The IDT, the IDE, the other tags and the DLC of the received remote frame are used for the reply.
3. When the transmission of the reply is completed the TXOK flag is set (interrupt).
4. All the parameters and data are available in the MOB until a new initialization.

16.5.2.5 Frame Buffer Receive Mode

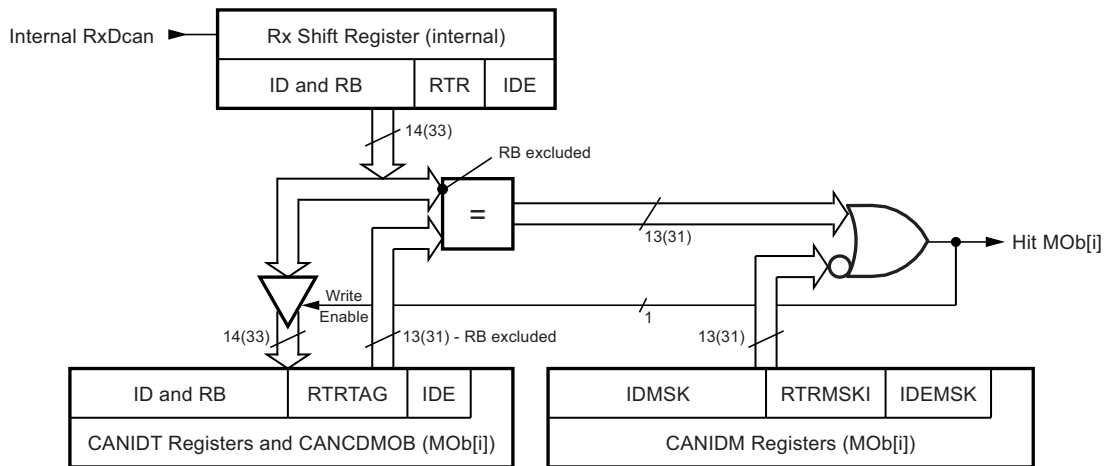
This mode is useful to receive multi frames. The priority between MObs offers a management for these incoming frames. One set MObs (including non-consecutive MObs) is created when the MObs are set in this mode. Due to the mode setting, only one set is possible. A frame buffer completed flag (or interrupt) - BXOK - will rise only when all the MObs of the set will have received their dedicated CAN frame.

1. MObs in frame buffer receive mode need to be initialized as MObs in standard receive mode.
2. The MObs are ready to receive data (or a remote) frames when their respective configurations are set (CONMOB).
3. When a frame identifier is received on CAN network, the CAN channel scans all the MObs in receive mode, tries to find the MOB having the highest priority which is matching.
4. On a hit, the IDT, the IDE and the DLC of the matched MOB are updated from the incoming (frame) values.
5. Once the reception is completed, the data bytes of the received message are stored (not for remote frame) in the data buffer of the matched MOB and the RXOK flag is set (interrupt).
6. When the reception in the last MOB of the set is completed, the frame buffer completed BXOK flag is set (interrupt). BXOK flag can be cleared only if all CONMOB fields of the set have been re-written before.
7. All the parameters and data are available in the MObs until a new initialization.

16.5.3 Acceptance Filter

Upon a reception hit (i.e., a good comparison between the ID + RTR + RBn + IDE received and an IDT + RTRTAG + RBnTAG + IDE specified while taking the comparison mask into account) the IDT + RTRTAG + RBnTAG + IDE received are updated in the MOB (written over the registers).

Figure 16-10. Acceptance Filter Block Diagram



Note: Examples:

Full filtering: to accept only ID = 0x317 in part A.

- ID MSK = 111 1111 1111_b
- ID TAG = 011 0001 0111_b

Partiel filtering: to accept ID from 0x310 up to 0x317 in part A.

- ID MSK = 111 1111 1000_b
- ID TAG = 011 0001 0xxx_b

No filtering: to accept all ID's from 0x000 up to 0x7FF in part A.

- ID MSK = 000 0000 0000_b
- ID TAG = xxx xxxx xxxx_b

16.5.4 MOB Page

Every MOB is mapped into a page to save place. The page number is the MOB number. This page number is set in CANPAGE register. The other numbers are reserved for factory tests.

CANHPMOB register gives the MOB having the highest priority in CANSIT registers. It is formatted to provide a direct entry for CANPAGE register. Because CANHPMOB codes CANSIT registers, it will be only updated if the corresponding enable bits (ENRX, ENTX, ENERR) are enabled (c.f. [Figure 16-14 on page 155](#)).

16.5.5 CAN Data Buffers

To preserve register allocation, the CAN data buffer is seen such as a FIFO (with address pointer accessible) into a MOB selection. This also allows to reduce the risks of un-controlled accesses.

There is one FIFO per MOB. This FIFO is accessed into a MOB page thanks to the CAN message register.

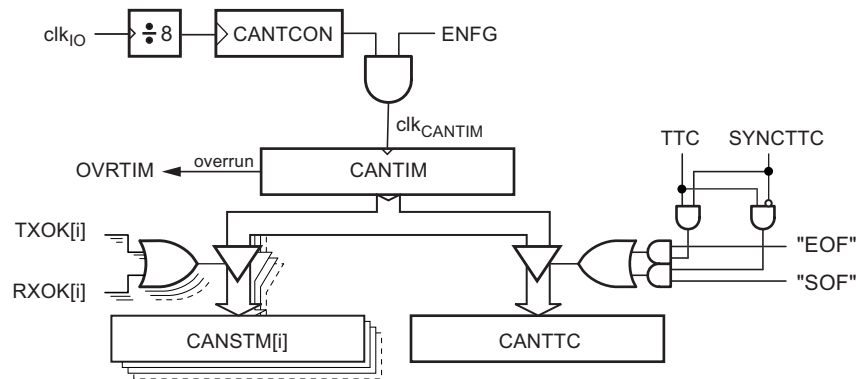
The data index (INDX) is the address pointer to the required data byte. The data byte can be read or write. The data index is automatically incremented after every access if the AINC* bit is reset. A roll-over is implemented, after data index=7 it is data index=0.

The first byte of a CAN frame is stored at the data index=0, the second one at the data index=1, ...

16.6 CAN Timer

A programmable 16-bit timer is used for message stamping and time trigger communication (TTC).

Figure 16-11. CAN Timer Block Diagram



16.6.1 Prescaler

An 8-bit prescaler is initialized by CANTCON register. It receives the clk_{IO} frequency divided by 8. It provides clk_{CANTIM} frequency to the CAN timer if the CAN controller is enabled.

$$T_{clk_{CANTIM}} = T_{clk_{IO}} \times 8 \times (CANTCON[7:0] + 1)$$

16.6.2 16-bit Timer

This timer starts counting from 0x0000 when the CAN controller is enabled (ENFG bit). When the timer rolls over from 0xFFFF to 0x0000, an interrupt is generated (OVRTIM).

16.6.3 Time Triggering

Two synchronization modes are implemented for TTC (TTC bit):

- synchronization on start of frame (SYNCTTC=0),
- synchronization on end of frame (SYNCTTC=1).

In TTC mode, a frame is sent once, even if an error occurs.

16.6.4 Stamping Message

The capture of the timer value is done in the MOB which receives or sends the frame. All managed MOB are stamped, the stamping of a received (sent) frame occurs on RxOk (TXOK).

16.7 Error Management

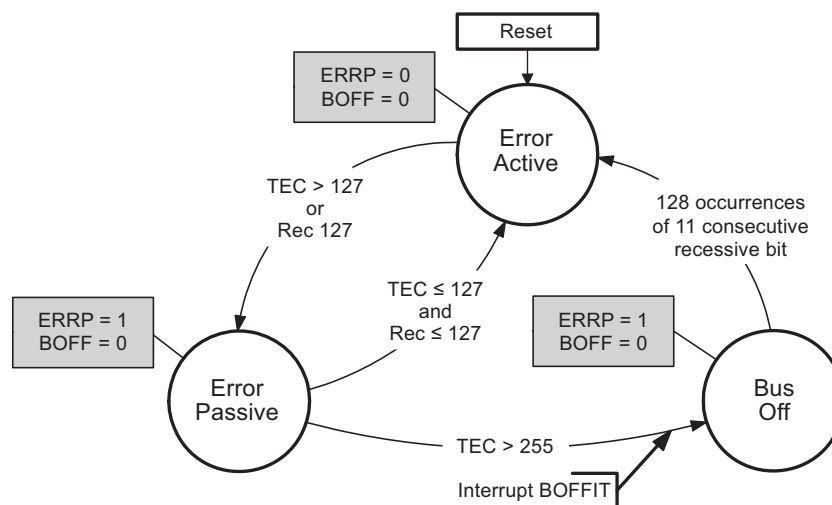
16.7.1 Fault Confinement

The CAN channel may be in one of the three following states:

- **Error active (default):**
The CAN channel takes part in bus communication and can send an active error frame when the CAN macro detects an error.
- **Error passive:**
The CAN channel cannot send an active error frame. It takes part in bus communication, but when an error is detected, a passive error frame is sent. Also, after a transmission, an error passive unit will wait before initiating further transmission.
- **Bus off:**
The CAN channel is not allowed to have any influence on the bus.

For fault confinement, a transmit error counter (TEC) and a receive error counter (REC) are implemented. BOFF and ERRP bits give the information of the state of the CAN channel. Setting BOFF to one may generate an interrupt.

Figure 16-12. Line Error Mode



Note: More than one REC/TEC change may apply during a given message transfer.

16.7.2 Error Types

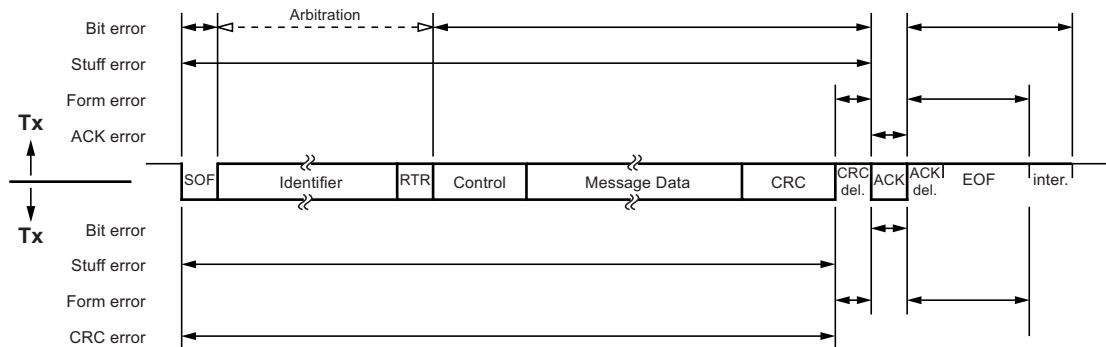
- **BERR**: Bit error. The bit value which is monitored is different from the bit value sent.

Note: Exceptions:

- Recessive bit sent monitored as dominant bit during the arbitration field and the acknowledge slot.
- Detecting a dominant bit during the sending of an error frame.

- **SERR**: Stuff error. Detection of more than five consecutive bit with the same polarity.
- **CERR**: CRC error (Rx only). The receiver performs a CRC check on every destuffed received message from the start of frame up to the data field. If this checking does not match with the destuffed CRC field, an CRC error is set.
- **FERR**: Form error. The form error results from one (or more) violations of the fixed form of the following bit fields:
 - CRC delimiter
 - acknowledgement delimiter
 - end-of-frame
 - error delimiter
 - overload delimiter
- **AERR**: Acknowledgment error (Tx only). No detection of the dominant bit in the acknowledge slot.

Figure 16-13. Error Detection Procedures in a Data Frame



16.7.3 Error Setting

The CAN channel can detect some errors on the CAN network.

- In transmission:
 - The error is set at MOB level.
- In reception:
 - The identified has matched:
 - The error is set at MOB level.
 - The identified has not or not yet matched:
 - The error is set at general level.

After detecting an error, the CAN channel sends an error frame on network. If the CAN channel detects an error frame on network, it sends its own error frame.

16.8 Interrupts

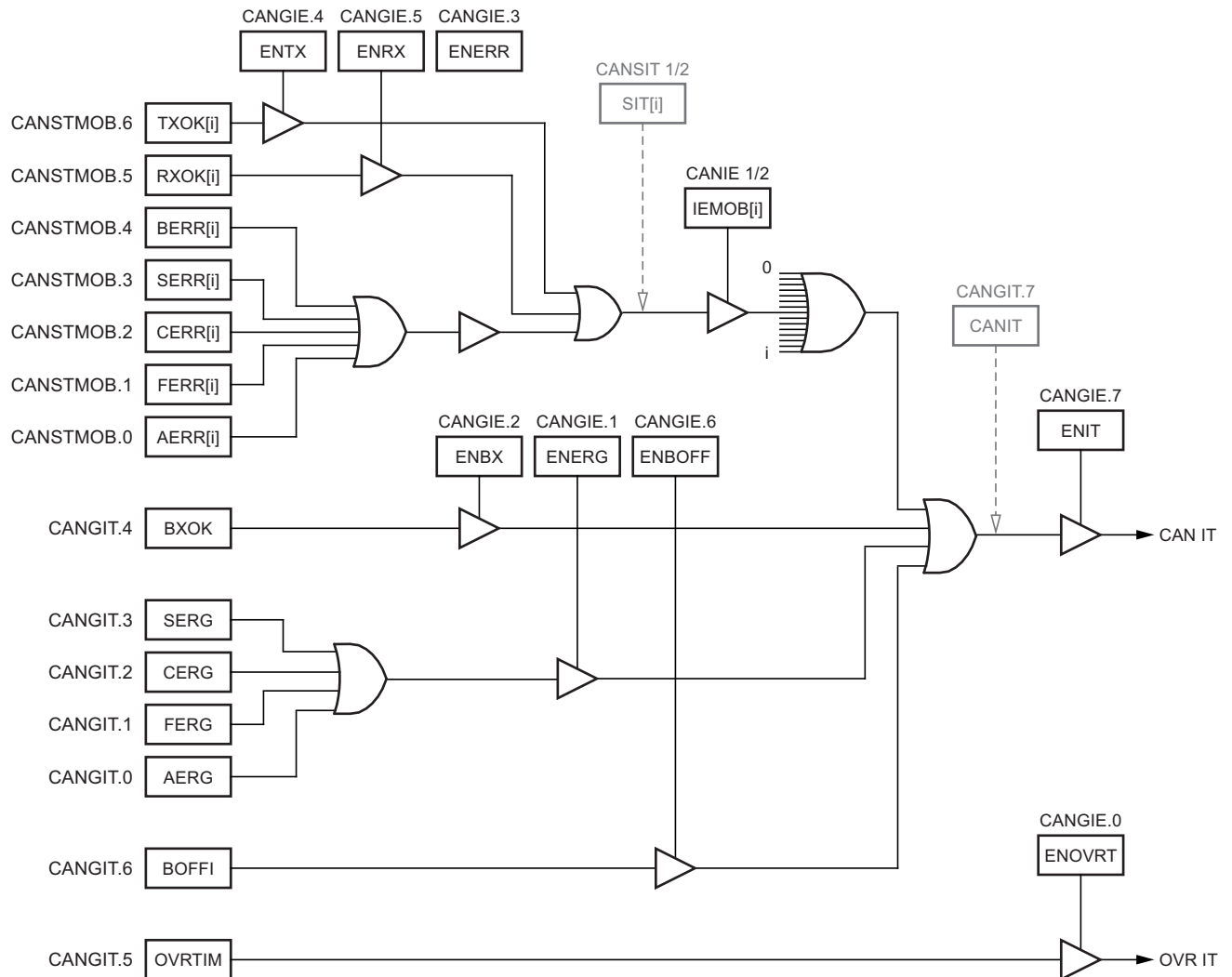
16.8.1 Interrupt organization

The different interrupts are:

- Interrupt on receive completed OK,
- Interrupt on transmit completed OK,
- Interrupt on error (bit error, stuff error, CRC error, form error, acknowledge error),
- Interrupt on frame buffer full,
- Interrupt on “Bus Off” setting,
- Interrupt on overrun of CAN timer.

The general interrupt enable is provided by ENIT bit and the specific interrupt enable for CAN timer overrun is provided by ENOVRT bit.

Figure 16-14. CAN Controller Interrupt Structure



16.8.2 Interrupt Behavior

When an interrupt occurs, an interrupt flag bit is set in the corresponding MOB-CANSTMOB register or in the general CANGIT register. If in the CANIE register, ENRX / ENTX / ENERR bit are set, then the corresponding MOB bit is set in the CANSITn register.

To acknowledge a MOB interrupt, the corresponding bits of CANSTMOB register (RXOK, TXOK,...) must be cleared by the software application. This operation needs a read-modify-write software routine.

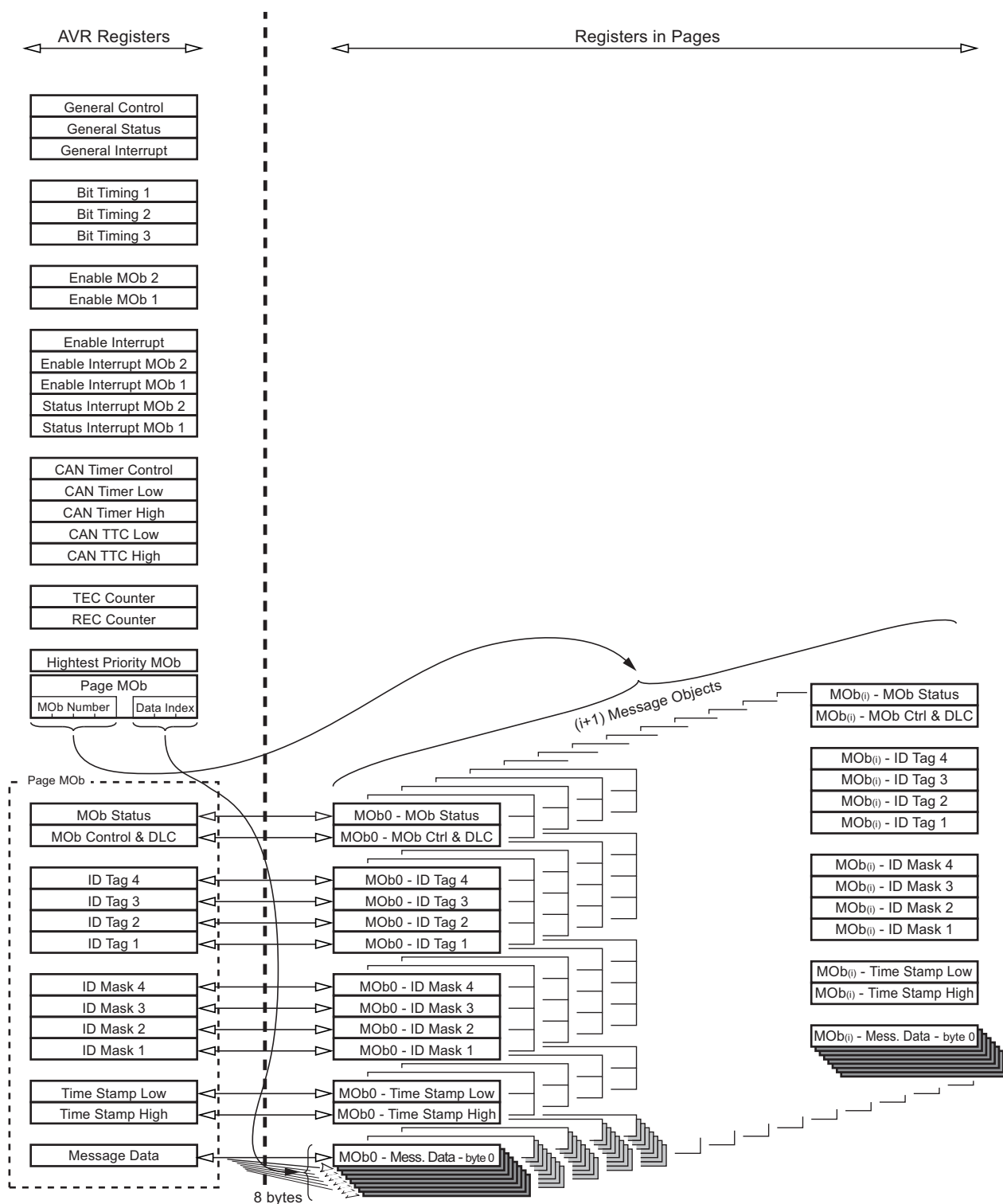
To acknowledge a general interrupt, the corresponding bits of CANGIT register (BXOK, BOFFIT,...) must be cleared by the software application. This operation is made writing a logical one in these interrupt flags (writing a logical zero doesn't change the interrupt flag value).

OVRTIM interrupt flag is reset as the other interrupt sources of CANGIT register and is also reset entering in its dedicated interrupt handler.

When the CAN node is in transmission and detects a Form Error in its frame, a bit Error will also be raised. Consequently, two consecutive interrupts can occur, both due to the same error. When a MOB error occurs and is set in its own CANSTMOB register, no general error is set in CANGIT register.

16.9 CAN Register Description

Figure 16-15. Registers Organization



16.10 General CAN Registers

16.10.1 CAN General Control Register - CANGCON

Bit	7	6	5	4	3	2	1	0	
	ABRQ	OVRQ	TTC	SYNTTC	LISTEN	TEST	ENA/STB	SWRES	CANGCON
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

• Bit 7 – ABRQ: Abort Request

This is not an auto resettable bit.

- 0 - no request.
- 1 - abort request: a reset of CANEN1 and CANEN2 registers is done. The pending communications are immediately disabled and the on-going one will be normally terminated, setting the appropriate status flags.
Note that CANCDMOB register remain unchanged.

• Bit 6 – OVRQ: Overload Frame Request

This is not an auto resettable bit.

- 0 - no request.
- 1 - overload frame request: send an overload frame after the next received frame.

The overload frame can be traced observing OVFG in CANGSTA register (c.f. [Figure 16-9 on page 149](#)).

• Bit 5 – TTC: Time Trigger Communication

- 0 - no TTC.
- 1 - TTC mode.

• Bit 4 – SYNTTC: Synchronization of TTC

This bit is only used in TTC mode.

- 0 - the TTC timer is caught on SOF.
- 1 - the TTC timer is caught on the last bit of the EOF.

• Bit 3 – LISTEN: Listening Mode

- 0 - no listening mode.
- 1 - listening mode.

• Bit 2 – TEST: Test Mode

- 0 - no test mode
- 1 - test mode: intend for factory testing and not for customer use.

Note: CAN may malfunction if this bit is set.

• Bit 1 – ENA/STB: Enable / Standby Mode

Because this bit is a command and is not immediately effective, the ENFG bit in CANGSTA register gives the true state of the chosen mode.

- 0 - standby mode: The on-going transmission (if exists) is normally terminated and the CAN channel is frozen (the CONMOB bits of every MOB do not change). The transmitter constantly provides a recessive level. In this mode, the receiver is not enabled but all the registers and mailbox remain accessible from CPU. In this mode, the receiver is not enabled but all the registers and mailbox remain accessible from CPU.

Note: A standby mode applied during a reception may corrupt the on-going reception or set the controller in a wrong state. The controller will restart correctly from this state if a software reset (SWRES) is applied. If no reset is considered, a possible solution is to wait for a lake of a receiver busy (RXBSY) before to enter in stand-by mode. The best solution is first to apply an abort request command (ABRQ) and then wait for the lake of the receiver busy (RXBSY) before to enter in stand-by mode. In any cases, this standby mode behavior has no effect on the CAN bus integrity.

- 1 - enable mode: The CAN channel enters in enable mode once 11 recessive bits has been read.

- **Bit 0 – SWRES: Software Reset Request**

This auto resettable bit only resets the CAN controller.

- 0 - no reset
- 1 - reset: this reset is “ORed” with the hardware reset.

16.10.2 CAN General Status Register - CANGSTA

Bit	7	6	5	4	3	2	1	0	
	-	OVRG	-	TXBSY	RXBSY	ENFG	BOFF	ERRP	CANGSTA
Read/Write	-	R	-	R	R	R	R	R	
Initial Value	-	0	-	0	0	0	0	0	

- **Bit 7 – Reserved Bit**

This bit is reserved for future use.

- **Bit 6 – OVRG: Overload Frame Flag**

This flag does not generate an interrupt.

- 0 - no overload frame.
- 1 - overload frame: set by hardware as long as the produced overload frame is sent.

- **Bit 5 – Reserved Bit**

This bit is reserved for future use.

- **Bit 4 – TXBSY: Transmitter Busy**

This flag does not generate an interrupt.

- 0 - transmitter not busy.
- 1 - transmitter busy: set by hardware as long as a frame (data, remote, overload or error frame) or an ACK field is sent. Also set when an inter frame space is sent.

- **Bit 3 – RXBSY: Receiver Busy**

This flag does not generate an interrupt.

- 0 - receiver not busy
- 1 - receiver busy: set by hardware as long as a frame is received or monitored.

- **Bit 2 – ENFG: Enable Flag**

This flag does not generate an interrupt.

- 0 - CAN controller disable: because an enable/standby command is not immediately effective, this status gives the true state of the chosen mode.
- 1 - CAN controller enable.

- **Bit 1 – BOFF: Bus Off Mode**

BOFF gives the information of the state of the CAN channel. Only entering in bus off mode generates the BOFFIT interrupt.

- 0 - no bus off mode.
- 1 - bus off mode.

- **Bit 0 – ERRP: Error Passive Mode**

ERRP gives the information of the state of the CAN channel. This flag does not generate an interrupt.

- 0 - no error passive mode.
- 1 - error passive mode.

16.10.3 CAN General Interrupt Register - CANGIT

Bit	7	6	5	4	3	2	1	0	
	CANIT	BOFFIT	OVRTIM	BXOK	SERG	CERG	FERG	AERG	CANGIT
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – CANIT: General Interrupt Flag**

This is a read only bit.

- 0 - no interrupt.
- 1 - CAN interrupt: image of all the CAN controller interrupts except for OVRTIM interrupt. This bit can be used for polling method.

- **Bit 6 – BOFFIT: Bus Off Interrupt Flag**

Writing a logical one resets this interrupt flag. BOFFIT flag is only set when the CAN enters in bus off mode (coming from error passive mode).

- 0 - no interrupt.
- 1 - bus off interrupt when the CAN enters in bus off mode.

- **Bit 5 – OVRTIM: Overrun CAN Timer**

Writing a logical one resets this interrupt flag. Entering in CAN timer overrun interrupt handler also reset this interrupt flag

- 0 - no interrupt.
- 1 - CAN timer overrun interrupt: set when the CAN timer switches from 0xFFFF to 0.

- **Bit 4 – BXOK: Frame Buffer Receive Interrupt**

Writing a logical one resets this interrupt flag. BXOK flag can be cleared only if all CONMOB fields of the MOB's of the buffer have been re-written before.

- 0 - no interrupt.
- 1 - burst receive interrupt: set when the frame buffer receive is completed.

- **Bit 3 – SERG: Stuff Error General**

Writing a logical one resets this interrupt flag.

- 0 - no interrupt.
- 1 - stuff error interrupt: detection of more than 5 consecutive bits with the same polarity.

- **Bit 2 – CERG: CRC Error General**

Writing a logical one resets this interrupt flag.

- 0 - no interrupt.
- 1 - CRC error interrupt: the CRC check on destuffed message does not fit with the CRC field.

- **Bit 1 – FERG: Form Error General**

Writing a logical one resets this interrupt flag.

- 0 - no interrupt.
- 1 - form error interrupt: one or more violations of the fixed form in the CRC delimiter, acknowledgment delimiter or EOF.

- **Bit 0 – AERG: Acknowledgment Error General**

Writing a logical one resets this interrupt flag.

- 0 - no interrupt.
- 1 - acknowledgment error interrupt: no detection of the dominant bit in acknowledge slot.

16.10.4 CAN General Interrupt Enable Register - CANGIE

Bit	7	6	5	4	3	2	1	0	
	ENIT	ENBOFF	ENRX	ENTX	ENERR	ENBX	ENERG	ENOVRT	CANGIE
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – ENIT: Enable all Interrupts** (Except for CAN Timer Overrun Interrupt)

- 0 - interrupt disabled.
- 1- CANIT interrupt enabled.

- **Bit 6 – ENBOFF: Enable Bus Off Interrupt**

- 0 - interrupt disabled.
- 1- bus off interrupt enabled.

- **Bit 5 – ENRX: Enable Receive Interrupt**

- 0 - interrupt disabled.
- 1- receive interrupt enabled.

- **Bit 4 – ENTX: Enable Transmit Interrupt**

- 0 - interrupt disabled.
- 1- transmit interrupt enabled.

- **Bit 3 – ENERR: Enable MOB Errors Interrupt**

- 0 - interrupt disabled.
- 1- MOB errors interrupt enabled.

- **Bit 2 – ENBX: Enable Frame Buffer Interrupt**

- 0 - interrupt disabled.
- 1- frame buffer interrupt enabled.

- **Bit 1 – ENERG: Enable General Errors Interrupt**

- 0 - interrupt disabled.
- 1- general errors interrupt enabled.

- **Bit 0 – ENOVRT: Enable CAN Timer Overrun Interrupt**

- 0 - interrupt disabled.
- 1- CAN timer interrupt overrun enabled.

16.10.5 CAN Enable MOB Registers - CANEN2 and CANEN1

Bit	7	6	5	4	3	2	1	0	
	-	-	ENMOB5	ENMOB4	ENMOB3	ENMOB2	ENMOB1	ENMOB0	CANEN2
	-	-	-	-	-	-	-	-	CANEN1
Bit	15	14	13	12	11	10	9	8	
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 5:0 - ENMOB5:0: Enable MOB**

This bit provides the availability of the MOB.

It is set to one when the MOB is enabled (i.e. CONMOB1:0 of CANCDMOB register).

Once TXOK or RXOK is set to one (TXOK for automatic reply), the corresponding ENMOB is reset. ENMOB is also set to zero configuring the MOB in disabled mode, applying abortion or standby mode.

- 0 - message object disabled: MOB available for a new transmission or reception.
- 1 - message object enabled: MOB in use.

- **Bit 15:6 – Reserved Bits**

These bits are reserved for future use.

16.10.6 CAN Enable Interrupt MOB Registers - CANIE2 and CANIE1

Bit	7	6	5	4	3	2	1	0	
	-	-	IEMOB5	IEMOB4	IEMOB3	IEMOB2	IEMOB1	IEMOB0	CANIE2
	-	-	-	-	-	-	-	-	CANIE1
Bit	15	14	13	12	11	10	9	8	
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 5:0 - IEMOB5:0: Interrupt Enable by MOB**

- 0 - interrupt disabled.
- 1 - MOB interrupt enabled

Note: Example: CANIE2 = 0000 1100_b: enable of interrupts on MOB 2 and 3.

- **Bit 15:6 – Reserved Bits**

These bits are reserved for future use. For compatibility with future devices, it must be written to zero when CANIE1 and CANIE2 are written.

16.10.7 CAN Status Interrupt MOB Registers - CANSIT2 and CANSIT1

Bit	7	6	5	4	3	2	1	0	
	-	-	SIT5	SIT4	SIT3	SIT2	SIT1	SIT0	CANSIT2
	-	-	-	-	-	-	-	-	CANSIT1
Bit	15	14	13	12	11	10	9	8	
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 5:0 - SIT5:0: Status of Interrupt by MOB**

- 0 - no interrupt.
- 1- MOB interrupt.

Note: Example: CANSIT2 = 0010 0001_b: MOB 0 and 5 interrupts.

- **Bit 15:6 – Reserved Bits**

These bits are reserved for future use.

16.10.8 CAN Bit Timing Register 1 - CANBT1

Bit	7	6	5	4	3	2	1	0	
	-	BRP5	BRP4	BRP3	BRP2	BRP1	BRP0	-	CANBT1
Read/Write	-	R/W	R/W	R/W	R/W	R/W	R/W	-	
Initial Value	-	0	0	0	0	0	0	-	

- **Bit 7– Reserved Bit**

This bit is reserved for future use. For compatibility with future devices, it must be written to zero when CANBT1 is written.

- **Bit 6:1 – BRP5:0: Baud Rate Prescaler**

The period of the CAN controller system clock T_{scl} is programmable and determines the individual bit timing.

$$T_{scl} = \frac{BRP[5:0] + 1}{clk_{IO} \text{ frequency}}$$

If 'BRP[5..0]=0', see [Section 16.4.3 “Baud Rate” on page 148](#) and [Section • “Bit 0 – SMP: Sample Point\(s\)” on page 164](#).

- **Bit 0 – Reserved Bit**

This bit is reserved for future use. For compatibility with future devices, it must be written to zero when CANBT1 is written.

16.10.9 CAN Bit Timing Register 2 - CANBT2

Bit	7	6	5	4	3	2	1	0	
	-	SJW1	SJW0	-	PRS2	PRS1	PRS0	-	CANBT2
Read/Write	-	R/W	R/W	-	R/W	R/W	R/W	-	
Initial Value	-	0	0	-	0	0	0	-	

- **Bit 7– Reserved Bit**

This bit is reserved for future use. For compatibility with future devices, it must be written to zero when CANBT2 is written.

- **Bit 6:5 – SJW1:0: Re-Synchronization Jump Width**

To compensate for phase shifts between clock oscillators of different bus controllers, the controller must re-synchronize on any relevant signal edge of the current transmission. The synchronization jump width defines the maximum number of clock cycles. A bit period may be shortened or lengthened by a re-synchronization.

$$T_{sjw} = T_{scl} \times (SJW[1:0] + 1)$$

- **Bit 4 – Reserved Bit**

This bit is reserved for future use. For compatibility with future devices, it must be written to zero when CANBT2 is written.

- **Bit 3:1 – PRS2:0: Propagation Time Segment**

This part of the bit time is used to compensate for the physical delay times within the network. It is twice the sum of the signal propagation time on the bus line, the input comparator delay and the output driver delay.

$$T_{prs} = T_{scl} \times (PRS[2:0] + 1)$$

- **Bit 0 – Reserved Bit**

This bit is reserved for future use. For compatibility with future devices, it must be written to zero when CANBT2 is written.

16.10.10 CAN Bit Timing Register 3 - CANBT3

Bit	7	6	5	4	3	2	1	0	
	-	PHS22	PHS21	PHS20	PHS12	PHS11	PHS10	SMP	CANBT3
Read/Write	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	-	0	0	0	0	0	0	0	

- **Bit 7– Reserved Bit**

This bit is reserved for future use. For compatibility with future devices, it must be written to zero when CANBT3 is written.

- **Bit 6:4 – PHS22:0: Phase Segment 2**

This phase is used to compensate for phase edge errors. This segment may be shortened by the re-synchronization jump width. PHS2[2..0] shall be ≥ 1 and $\leq PHS1[2..0]$ (c.f. [Section 16.2.3 “CAN Bit Timing” on page 143](#) and [Section 16.4.3 “Baud Rate” on page 148](#)).

$$T_{phs2} = T_{scl} \times (PHS2[2:0] + 1)$$

- **Bit 3:1 – PHS12:0: Phase Segment 1**

This phase is used to compensate for phase edge errors. This segment may be lengthened by the re-synchronization jump width.

$$T_{phs1} = T_{scl} \times (PHS1[2:0] + 1)$$

- **Bit 0 – SMP: Sample Point(s)**

This option allows to filter possible noise on TxCAN input pin.

- 0 - the sampling will occur once at the user configured sampling point - **SP**.
- 1 - with three-point sampling configuration the first sampling will occur two $T_{clk_{IO}}$ clocks before the user configured sampling point - **SP**, again at one $T_{clk_{IO}}$ clock before **SP** and finally at **SP**. Then the bit level will be determined by a majority vote of the three samples.

‘SMP=1’ configuration is not compatible with ‘BRP[5:0]=0’ because $T_Q = T_{clk_{IO}}$.

If BRP = 0, SMP must be cleared.

16.10.11 CAN Timer Control Register - CANTCON

Bit	7	6	5	4	3	2	1	0	
	TPRSC7	TPRSC6	TPRSC5	TPRSC4	TPRSC3	TPRSC2	TPRSC1	TPRSC0	CANTCON
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:0 – TPRSC7:0: CAN Timer Prescaler**

Prescaler for the CAN timer upper counter range 0 to 255. It provides the clock to the CAN timer if the CAN controller is enabled.

$$T_{clk_{CANTIM}} = T_{clk_{IO}} \times 8 \times (CANTCON [7:0] + 1)$$

16.10.12 CAN Timer Registers - CANTIML and CANTIMH

Bit	7	6	5	4	3	2	1	0	
	CANTIM7	CANTIM6	CANTIM5	CANTIM4	CANTIM3	CANTIM2	CANTIM1	CANTIM0	CANTIM L
	CANTIM15	CANTIM14	CANTIM13	CANTIM12	CANTIM11	CANTIM10	CANTIM9	CANTIM8	CANTIM H
Bit	15	14	13	12	11	10	9	8	
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 15:0 - CANTIM15:0: CAN Timer Count**

CAN timer counter range 0 to 65,535.

16.10.13 CAN TTC Timer Registers - CANTTCL and CANTTCH

Bit	7	6	5	4	3	2	1	0	
	TIMTTC7	TIMTTC6	TIMTTC5	TIMTTC4	TIMTTC3	TIMTTC2	TIMTTC1	TIMTTC0	CANTTCL
	TIMTTC15	TIMTTC14	TIMTTC13	TIMTTC12	TIMTTC11	TIMTTC10	TIMTTC9	TIMTTC8	CANTTCH
Bit	15	14	13	12	11	10	9	8	
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bits 15:0 - TIMTTC15:0: TTC Timer Count**

CAN TTC timer counter range 0 to 65,535.

16.10.14 CAN Transmit Error Counter Register - CANTEC

Bit	7	6	5	4	3	2	1	0	
	TEC7	TEC6	TEC5	TEC4	TEC3	TEC2	TEC1	TEC0	CANTEC
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:0 – TEC7:0: Transmit Error Count**

CAN transmit error counter range 0 to 255.

16.10.15 CAN Receive Error Counter Register - CANREC

Bit	7	6	5	4	3	2	1	0	
	REC7	REC6	REC5	REC4	REC3	REC2	REC1	REC0	CANREC
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:0 – REC7:0: Receive Error Count**

CAN receive error counter range 0 to 255.

16.10.16 CAN Highest Priority MOB Register - CANHPMOB

Bit	7	6	5	4	3	2	1	0	
	HPMOB3	HPMOB2	HPMOB1	HPMOB0	CGP3	CGP2	CGP1	CGP0	CANHPMOB
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	1	1	1	1	0	0	0	0	

- **Bit 7:4 – HPMOB3:0: Highest Priority MOB Number**

MOB having the highest priority in CANSIT registers.

If CANSIT = 0 (no MOB), the return value is 0xF.

Note: Do not confuse “MOB priority” and “Message ID priority” - <Helv>See “Message Objects” on page 149.

- **Bit 3:0 – CGP3:0: CAN General Purpose Bits**

These bits can be pre-programmed to match with the wanted configuration of the CANPAGE register (i.e., $\overline{\text{AINC}}$ and INDX2:0 setting).

16.10.17 CAN Page MOB Register - CANPAGE

Bit	7	6	5	4	3	2	1	0	
	MOBNB3	MOBNB2	MOBNB1	MOBNB0	AINC	INDX2	INDX1	INDX0	CANPAGE
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:4 – MOBNB3:0: MOB Number**

Selection of the MOB number, the available numbers are from 0 to 5.

Note: MOBNB3 always must be written to zero for compatibility with all AVR CAN devices.

- **Bit 3 – $\overline{\text{AINC}}$: Auto Increment of the FIFO CAN Data Buffer Index (Active Low)**

- 0 - auto increment of the index (default value).
- 1- no auto increment of the index.

- **Bit 2:0 – INDX2:0: FIFO CAN Data Buffer Index**

Byte location of the CAN data byte into the FIFO for the defined MOB.

16.11 MOB Registers

The MOB registers has **no** initial (default) value after RESET.

16.11.1 CAN MOB Status Register - CANSTMOB

Bit	7	6	5	4	3	2	1	0	
	DLCW	TXOK	RXOK	BERR	SERR	CERR	FERR	AERR	CANSTMOB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	-	-	-	-	-	-	-	-	

- **Bit 7 – DLCW: Data Length Code Warning**

The incoming message does not have the DLC expected. Whatever the frame type, the DLC field of the CANCDMOB register is updated by the received DLC.

- **Bit 6 – TXOK: Transmit OK**

This flag can generate an interrupt. It must be cleared using a read-modify-write software routine on the whole CANSTMOB register.

The communication enabled by transmission is completed. TxOK rises at the end of EOF field. When the controller is ready to send a frame, if two or more message objects are enabled as producers, the lower MOB index (0 to 14) is supplied first.

- **Bit 5 – RXOK: Receive OK**

This flag can generate an interrupt. It must be cleared using a read-modify-write software routine on the whole CANSTMOB register.

The communication enabled by reception is completed. RxOK rises at the end of the 6th bit of EOF field. In case of two or more message object reception hits, the lower MOB index (0 to 14) is updated first.

- **Bit 4 – BERR: Bit Error (Only in Transmission)**

This flag can generate an interrupt. It must be cleared using a read-modify-write software routine on the whole CANSTMOB register.

The bit value monitored is different from the bit value sent.

Exceptions: the monitored recessive bit sent as a dominant bit during the arbitration field and the acknowledge slot detecting a dominant bit during the sending of an error frame.

- **Bit 3 – SERR: Stuff Error**

This flag can generate an interrupt. It must be cleared using a read-modify-write software routine on the whole CANSTMOB register.

Detection of more than five consecutive bits with the same polarity. This flag can generate an interrupt.

- **Bit 2 – CERR: CRC Error**

This flag can generate an interrupt. It must be cleared using a read-modify-write software routine on the whole CANSTMOB register.

The receiver performs a CRC check on every de-stuffed received message from the start of frame up to the data field. If this checking does not match with the de-stuffed CRC field, a CRC error is set.

- **Bit 1 – FERR: Form Error**

This flag can generate an interrupt. It must be cleared using a read-modify-write software routine on the whole CANSTMOB register.

The form error results from one or more violations of the fixed form in the following bit fields:

- CRC delimiter.
- Acknowledgment delimiter.
- EOF

- **Bit 0 – AERR: Acknowledgment Error**

This flag can generate an interrupt. It must be cleared using a read-modify-write software routine on the whole CANSTMOB register.

No detection of the dominant bit in the acknowledge slot.

16.11.2 CAN MOB Control and DLC Register - CANCDMOB

Bit	7	6	5	4	3	2	1	0	
	CONMOB 1	CONMOB 0	RPLV	IDE	DLC3	DLC2	DLC1	DLC0	CANCDMOB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	-	-	-	-	-	-	-	-	

- **Bit 7:6 – CONMOB1:0: Configuration of Message Object**

These bits set the communication to be performed (**no** initial value after RESET).

- 00 - disable.
- 01 - enable transmission.
- 10 - enable reception.
- 11 - enable frame buffer reception

These bits are **not** cleared once the communication is performed. The user must re-write the configuration to enable a new communication.

- This operation is necessary to be able to reset the BXOK flag.
- This operation also set the corresponding bit in the CANEN registers.

- **Bit 5 – RPLV: Reply Valid**

Used in the automatic reply mode after receiving a remote frame.

- 0 - reply not ready.
- 1 - reply ready and valid.

- **Bit 4 – IDE: Identifier Extension**

IDE bit of the remote or data frame to send.

This bit is updated with the corresponding value of the remote or data frame received.

- 0 - CAN standard rev 2.0 A (identifiers length = 11 bits).
- 1 - CAN standard rev 2.0 B (identifiers length = 29 bits).

- **Bit 3:0 – DLC3:0: Data Length Code**

Number of Bytes in the data field of the message.

DLC field of the remote or data frame to send. The range of DLC is from 0 up to 8. If DLC field >8 then effective DLC=8.

This field is updated with the corresponding value of the remote or data frame received. If the expected DLC differs from the incoming DLC, a DLC warning appears in the CANSTMOB register.

16.11.3 CAN Identifier Tag Registers - CANIDT1, CANIDT2, CANIDT3, and CANIDT4

V2.0 part A								
Bit	15/7	14/6	13/5	12/4	11/3	10/2	9/1	8/0
	-	-	-	-	-	RTRTAG	-	RB0TAG
	-	-	-	-	-	-	-	-
	IDT2	IDT1	IDT0	-	-	-	-	-
	IDT10	IDT9	IDT8	IDT7	IDT6	IDT5	IDT4	IDT3
Bit	31/23	30/22	29/21	28/20	27/19	26/18	25/17	24/16
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	-	-	-	-	-	-	-	-
V2.0 part B								
Bit	15/7	14/6	13/5	12/4	11/3	10/2	9/1	8/0
	IDT4	IDT3	IDT2	IDT1	IDT0	RTRTAG	RB1TAG	RB0TAG
	IDT12	IDT11	IDT10	IDT9	IDT8	IDT7	IDT6	IDT5
	IDT20	IDT19	IDT18	IDT17	IDT16	IDT15	IDT14	IDT13
	IDT28	IDT27	IDT26	IDT25	IDT24	IDT23	IDT22	IDT21
Bit	31/23	30/22	29/21	28/20	27/19	26/18	25/17	24/16
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	-	-	-	-	-	-	-	-

16.11.3.1 V2.0 part A

- **Bit 31:21 – IDT10:0: Identifier Tag**

Identifier field of the remote or data frame to send.

This field is updated with the corresponding value of the remote or data frame received.

- **Bit 20:3 – Reserved Bits**

These bits are reserved for future use. For compatibility with future devices, they must be written to zero when CANIDTn are written.

When a remote or data frame is received, these bits do not operate in the comparison but they are updated with un-predicted values.

- **Bit 2 – RTRTAG: Remote Transmission Request Tag**

RTR bit of the remote or data frame to send.

This tag is updated with the corresponding value of the remote or data frame received. In case of Automatic Reply mode, this bit is automatically reset before sending the response.

- **Bit 1 – Reserved Bit**

This bit is reserved for future use. For compatibility with future devices, it must be written to zero when CANIDTn are written.

When a remote or data frame is received, this bit does not operate in the comparison but it is updated with un-predicted values.

- **Bit 0 – RB0TAG: Reserved Bit 0 Tag**

RB0 bit of the remote or data frame to send.

This tag is updated with the corresponding value of the remote or data frame received.

16.11.3.2 V2.0 part B

- **Bit 31:3 – IDT28:0: Identifier Tag**

Identifier field of the remote or data frame to send.

This field is updated with the corresponding value of the remote or data frame received.

- **Bit 2 – RTRTAG: Remote Transmission Request Tag**

RTR bit of the remote or data frame to send.

This tag is updated with the corresponding value of the remote or data frame received. In case of Automatic Reply mode, this bit is automatically reset before sending the response.

- **Bit 1 – RB1TAG: Reserved Bit 1 Tag**

RB1 bit of the remote or data frame to send.

This tag is updated with the corresponding value of the remote or data frame received.

- **Bit 0 – RB0TAG: Reserved Bit 0 Tag**

RB0 bit of the remote or data frame to send.

This tag is updated with the corresponding value of the remote or data frame received.

16.11.4 CAN Identifier Mask Registers - CANIDM1, CANIDM2, CANIDM3, and CANIDM4

V2.0 part A

Bit	15/7	14/6	13/5	12/4	11/3	10/2	9/1	8/0	
	-	-	-	-	-	RTRMSK	-	IDEMSK	CANIDM4
	-	-	-	-	-	-	-	-	CANIDM3
	IDMSK2	IDMSK1	IDMSK0	-	-	-	-	-	CANIDM2
	IDMSK10	IDMSK9	IDMSK8	IDMSK7	IDMSK6	IDMSK5	IDMSK4	IDMSK3	CANIDM1
Bit	31/23	30/22	29/21	28/20	27/19	26/18	25/17	24/16	
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	-	-	-	-	-	-	-	-	

V2.0 part B

Bit	15/7	14/6	13/5	12/4	11/3	10/2	9/1	8/0	
	IDMSK4	IDMSK3	IDMSK2	IDMSK1	IDMSK0	RTRMSK	-	IDEMSK	CANIDM4
	IDMSK12	IDMSK11	IDMSK10	IDMSK9	IDMSK8	IDMSK7	IDMSK6	IDMSK5	CANIDM3
	IDMSK20	IDMSK19	IDMSK18	IDMSK17	IDMSK16	IDMSK15	IDMSK14	IDMSK13	CANIDM2
	IDMSK28	IDMSK27	IDMSK26	IDMSK25	IDMSK24	IDMSK23	IDMSK22	IDMSK21	CANIDM1
Bit	31/23	30/22	29/21	28/20	27/19	26/18	25/17	24/16	
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	-	-	-	-	-	-	-	-	

16.11.4.1 V2.0 part A

- **Bit 31:21 – IDMSK10:0: Identifier Mask**

- 0 - comparison true forced - see [Section 16.5.3 “Acceptance Filter” on page 151](#)
- 1 - bit comparison enabled - see [Section 16.5.3 “Acceptance Filter” on page 151](#)

- **Bit 20:3 – Reserved Bits**

These bits are reserved for future use. For compatibility with future devices, they must be written to zero when CANIDMn are written.

- **Bit 2 – RTRMSK: Remote Transmission Request Mask**

- 0 - comparison true forced
- 1 - bit comparison enabled.

- **Bit 1 – Reserved Bit**

This bit is reserved for future use. For compatibility with future devices, it must be written to zero when CANIDTn are written.

- **Bit 0 – IDEMSK: Identifier Extension Mask**

- 0 - comparison true forced
- 1 - bit comparison enabled.

16.11.4.2 V2.0 part B

- **Bit 31:3 – IDMSK28:0: Identifier Mask**

- 0 - comparison true forced - see [Section 16.5.3 “Acceptance Filter” on page 151](#)
- 1 - bit comparison enabled. - see [Section 16.5.3 “Acceptance Filter” on page 151](#)

- **Bit 2 – RTRMSK: Remote Transmission Request Mask**

- 0 - comparison true forced
- 1 - bit comparison enabled.

- **Bit 1 – Reserved Bit**

Writing zero in this bit is recommended.

- **Bit 0 – IDEMSK: Identifier Extension Mask**

- 0 - comparison true forced
- 1 - bit comparison enabled.

16.11.5 CAN Time Stamp Registers - CANSTML and CANSTMH

Bit	7	6	5	4	3	2	1	0	
	TIMSTM7	TIMSTM6	TIMSTM5	TIMSTM4	TIMSTM3	TIMSTM2	TIMSTM1	TIMSTM0	CANSTML
	TIMSTM15	TIMSTM14	TIMSTM13	TIMSTM12	TIMSTM11	TIMSTM10	TIMSTM9	TIMSTM8	CANSTMH
Bit	15	14	13	12	11	10	9	8	
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	-	-	-	-	-	-	-	-	

- **Bits 15:0 - TIMSTM15:0: Time Stamp Count**

CAN time stamp counter range 0 to 65,535.

16.11.6 CAN Data Message Register - CANMSG

Bit	7	6	5	4	3	2	1	0	
	MSG 7	MSG 6	MSG 5	MSG 4	MSG 3	MSG 2	MSG 1	MSG 0	CANMSG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	-	-	-	-	-	-	-	-	

- **Bit 7:0 – MSG7:0: Message Data**

This register contains the CAN data byte pointed at the page MOB register.

After writing in the page MOB register, this byte is equal to the specified message location of the pre-defined identifier + index. If auto-incrementation is used, at the end of the data register writing or reading cycle, the index is auto-incremented. The range of the counting is 8 with no end of loop (0, 1,..., 7, 0,...).

16.12 Examples of CAN Baud Rate Setting

The CAN bus requires very accurate timing especially for high baud rates. It is recommended to use only an external crystal for CAN operations.

(Refer to [Section 16.4.2 “Bit Timing” on page 147](#) and [Section 16.4.3 “Baud Rate” on page 148](#) for timing description and [Section 16.10.8 “CAN Bit Timing Register 1 - CANBT1” on page 163](#) to [Section 16.10.10 “CAN Bit Timing Register 3 - CANBT3” on page 164](#) for “CAN Bit Timing Registers”).

Table 16-2. Examples of CAN Baud Rate Settings for Commonly Frequencies

fCLK _{IO} (MHz)	CAN Rate (Kbps)	Description			Segments				Registers		
		Sampling Point	TQ (μs)	Tbit (TQ)	Tprs (TQ)	Tph1 (TQ)	Tph2 (TQ)	Tsjw (TQ)	CANBT1	CANBT2	CANBT3
16.000	1000	69% ⁽¹⁾	0.0625	16	7	4	4	1	0x00	0x0C	0x36 ⁽²⁾
		75%	0.125	8	3	2	2	1	0x02	0x04	0x13
	500	75%	0.125	16	7	4	4	1	0x02	0x0C	0x37
			0.250	8	3	2	2	1	0x06	0x04	0x13
	250	75%	0.250	16	7	4	4	1	0x06	0x0C	0x37
			0.500	8	3	2	2	1	0x0E	0x04	0x13
	200	75%	0.3125	16	7	4	4	1	0x08	0x0C	0x37
			0.625	8	3	2	2	1	0x12	0x04	0x13
	125	75%	0.500	16	7	4	4	1	0x0E	0x0C	0x37
			1.000	8	3	2	2	1	0x1E	0x04	0x13
	100	75%	0.625	16	7	4	4	1	0x12	0x0C	0x37
			1.250	8	3	2	2	1	0x26	0x04	0x13
12.000	1000	67% ⁽¹⁾	0.083333	12	5	3	3	1	0x00	0x08	0x24 ⁽²⁾
			x	- - - no data - - -							
	500	75%	0.166666	12	5	3	3	1	0x02	0x08	0x25
			0.250	8	3	2	2	1	0x04	0x04	0x13
	250	75%	0.250	16	7	4	4	1	0x04	0x0C	0x37
			0.500	8	3	2	2	1	0x0A	0x04	0x13
	200	75%	0.250	20	8	6	5	1	0x04	0x0E	0x4B
			0.416666	12	5	3	3	1	0x08	0x08	0x25
	125	75%	0.500	16	7	4	4	1	0x0A	0x0C	0x37
			1.000	8	3	2	2	1	0x16	0x04	0x13
	100	75%	0.500	20	8	6	5	1	0x0A	0x0E	0x4B
			0.833333	12	5	3	3	1	0x12	0x08	0x25

- Notes: 1. See [Section 16.4.3 “Baud Rate” on page 148](#).
2. See [Section • “Bit 0 – SMP: Sample Point\(s\)” on page 164](#)

Table 16-2. Examples of CAN Baud Rate Settings for Commonly Frequencies (Continued)

fCLK _{IO} (MHz)	CAN Rate (Kbps)	Description			Segments				Registers		
		Sampling Point	TQ (μs)	Tbit (TQ)	Tprs (TQ)	Tph1 (TQ)	Tph2 (TQ)	Tsjw (TQ)	CANBT1	CANBT2	CANBT3
8.000	1000	63% ⁽¹⁾		x	- - - no data - - -						
			0.125	8	3	2	2	1	0x00	0x04	0x12 ⁽²⁾
	500	69% ⁽¹⁾	0.125	16	7	4	4	1	0x00	0x0C	0x36 ⁽²⁾
			0.250	8	3	2	2	1	0x02	0x04	0x13
	250	75%	0.250	16	7	4	4	1	0x02	0x0C	0x37
			0.500	8	3	2	2	1	0x06	0x04	0x13
	200	75%	0.250	20	8	6	5	1	0x02	0x0E	0x4B
			0.625	8	3	2	2	1	0x08	0x04	0x13
	125	75%	0.500	16	7	4	4	1	0x06	0x0C	0x37
			1.000	8	3	2	2	1	0x0E	0x04	0x13
	100	75%	0.625	16	7	4	4	1	0x08	0x0C	0x37
			1.250	8	3	2	2	1	0x12	0x04	0x13
6.000	1000		- - - not applicable - - -								
	500	67% ⁽¹⁾	0.166666	12	5	3	3	1	0x00	0x08	0x24 ⁽²⁾
				x	- - - no data - - -						
	250	75%	0.333333	12	5	3	3	1	0x02	0x08	0x25
			0.500	8	3	2	2	1	0x04	0x04	0x13
	200	80%	0.333333	15	7	4	3	1	0x02	0x0C	0x35
			0.500	10	4	3	2	1	0x04	0x06	0x23
	125	75%	0.500	16	7	4	4	1	0x04	0x0C	0x37
			1.000	8	3	2	2	1	0x0A	0x04	0x13
	100	75%	0.500	20	8	6	5	1	0x04	0x0E	0x4B
			0.833333	12	5	3	3	1	0x08	0x08	0x25
4.000	1000		- - - not applicable - - -								
	500	63% ⁽¹⁾		x	- - - no data - - -						
			0.250	8	3	2	2	1	0x00	0x04	0x12 ⁽²⁾
	250	69% ⁽¹⁾	0.250	16	7	4	4	1	0x00	0x0C	0x36 ⁽²⁾
			0.500	8	3	2	2	1	0x02	0x04	0x13
	200	70% ⁽¹⁾	0.250	20	8	6	5	1	0x00	0x0E	0x4A ⁽²⁾
				x	- - - no data - - -						
	125	75%	0.500	16	7	4	4	1	0x02	0x0C	0x37
			1.000	8	3	2	2	1	0x06	0x04	0x13
	100	75%	0.500	20	8	6	5	1	0x02	0x0E	0x4B
			1.250	8	3	2	2	1	0x08	0x04	0x13

Notes: 1. See [Section 16.4.3 “Baud Rate” on page 148](#).
2. See [Section • “Bit 0 – SMP: Sample Point\(s\)” on page 164](#)